Universität Konstanz

Introduction to Computing for the Social Sciences

Session 12 – Parallel Programming

David Garcia

Winter Semester 2026-2027

Course Outline

- Session 1: Introduction
- Session 2: Information Coding
- Session 3: Data Structures
- Session 4: Software
- Session 5: Algorithms
- Session 6: Recursion
- Session 7: Sorting Algorithms
- Session 8: Time Complexity
- Session 9: Theoretical Computer Science
- Session 10: Machine Learning
- Session 11: Databases
- Session 12: Parallel Programming
  - Exercise: JQ + Parallelization exercises
- Session 13: Social Science Applications
- Session 14: Exam Review

Install jq for next exercise session

https://jqlang.org/



Overview

- Parallelization
- Hardware Parallelization
- Software Parallelization
- Limitations

Your laptop stopped getting faster in 2005

Universität Konstanz

MOTIVATING EXAMPLE

For thirty years, slow code got faster by waiting: the same program ran twice as quickly on the hardware of two years later. Then clock speeds flattened, around **3 GHz, where they still are**, while the transistor count kept doubling. The extra transistors went into **more cores, not faster ones**.

~3 GHz
clock speed, since about 2005

2 → 128+
cores per chip in the same period

0×
automatic speedup for sequential code

The free lunch is over. Making the extra cores useful is now a software problem, which means it is yours.

Sixteen cores, and 2.5× faster

Universität Konstanz

MOTIVATING EXAMPLE

A simulation takes **40 hours**. You move it to a 16-core machine expecting two and a half. It takes **16 hours**. You move it to **64 cores**. It takes **15**.

2.5×
faster, on 16 cores

2.7×
faster, on 64 cores

2.8×
the ceiling, at any number of cores

Where did the other thirteen cores go? There is an exact formula, and it was written down in 1967.

Three billion posts, one million rows of memory

Universität Konstanz

MOTIVATING EXAMPLE

You are asked for the **correlation** between likes and reshares across **3 billion** posts. Your machine holds **one million** at a time. Computing the correlation on each chunk and averaging gives a number, and it is **the wrong number**.

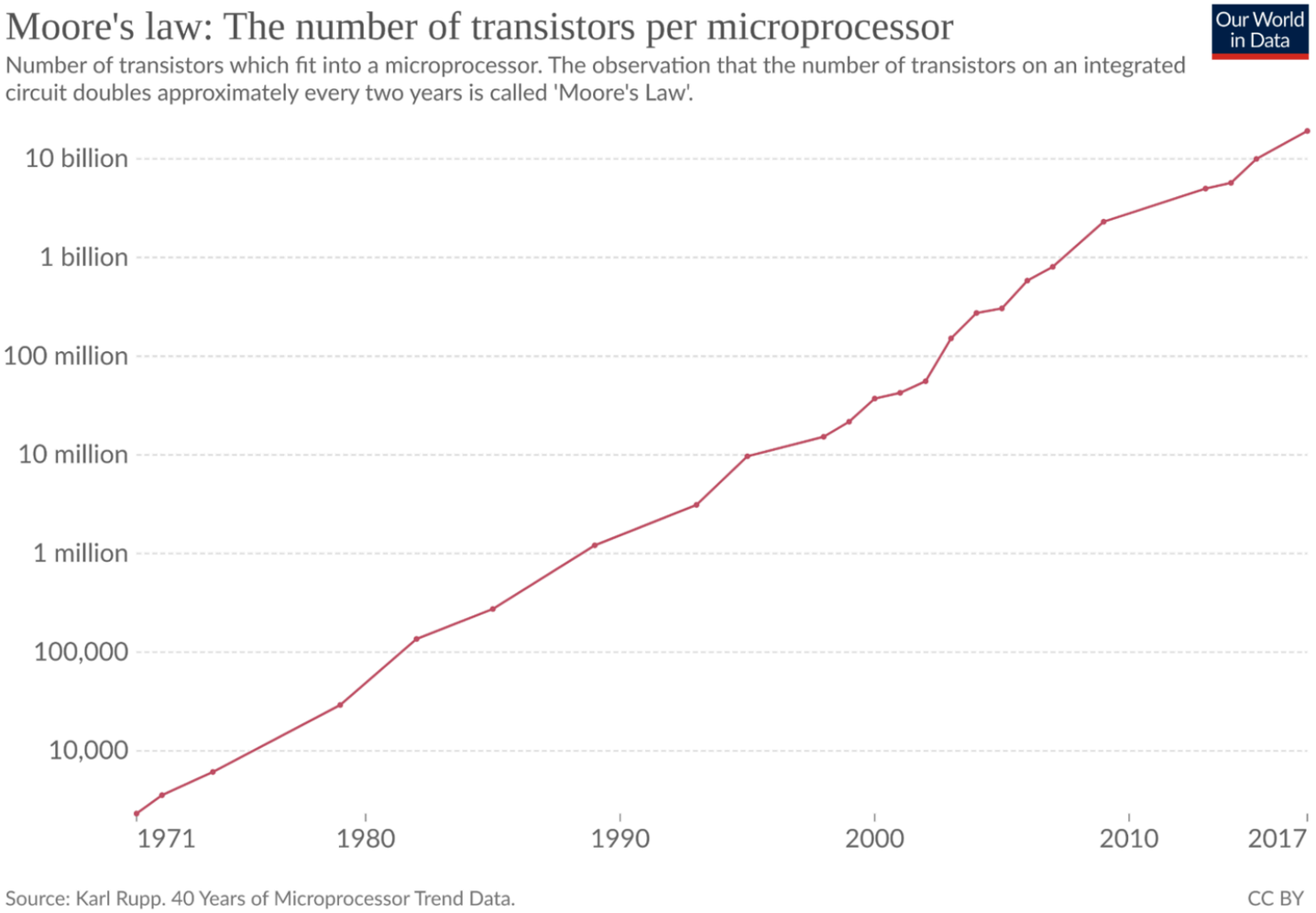
3×10^9
posts

10^6
fit in memory at once

6
numbers per chunk are enough for the exact answer

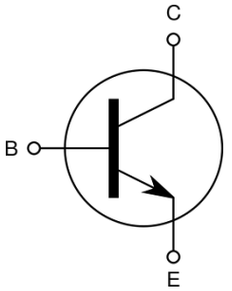
Which statistics can be split across machines and reassembled exactly, and which cannot?

Moore’s law


Moore's law: The number of transistors per microprocessor
Number of transistors which fit into a microprocessor. The observation that the number of transistors on an integrated circuit doubles approximately every two years is called 'Moore's Law'.
Our World in Data
10 billion
1 billion
100 million
10 million
1 million
100,000
10,000
1971
1980
1990
2000
2010
2017
Source: Karl Rupp. 40 Years of Microprocessor Trend Data.
CC BY

Moore’s law and its limits

- **Moore’s law** predicts that the number of transistors per microchip doubles every two years – exponential growth in computing power
- While the increase is still exponential, the rate is slower and doubling takes more than two years
- There is a **physical limit**: a transistor cannot be smaller than an atom
- The limit is expected to be reached in this decade unless radically different models of computation become feasible
- **Parallelization is a workaround to this limit:** using several processors

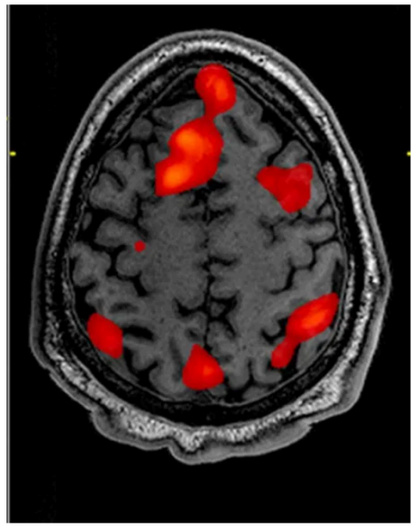




To learn more about Moore’s law: https://ourworldindata.org/technological-change#advances-in-computational-technology

The brain as an inspiration for parallelization

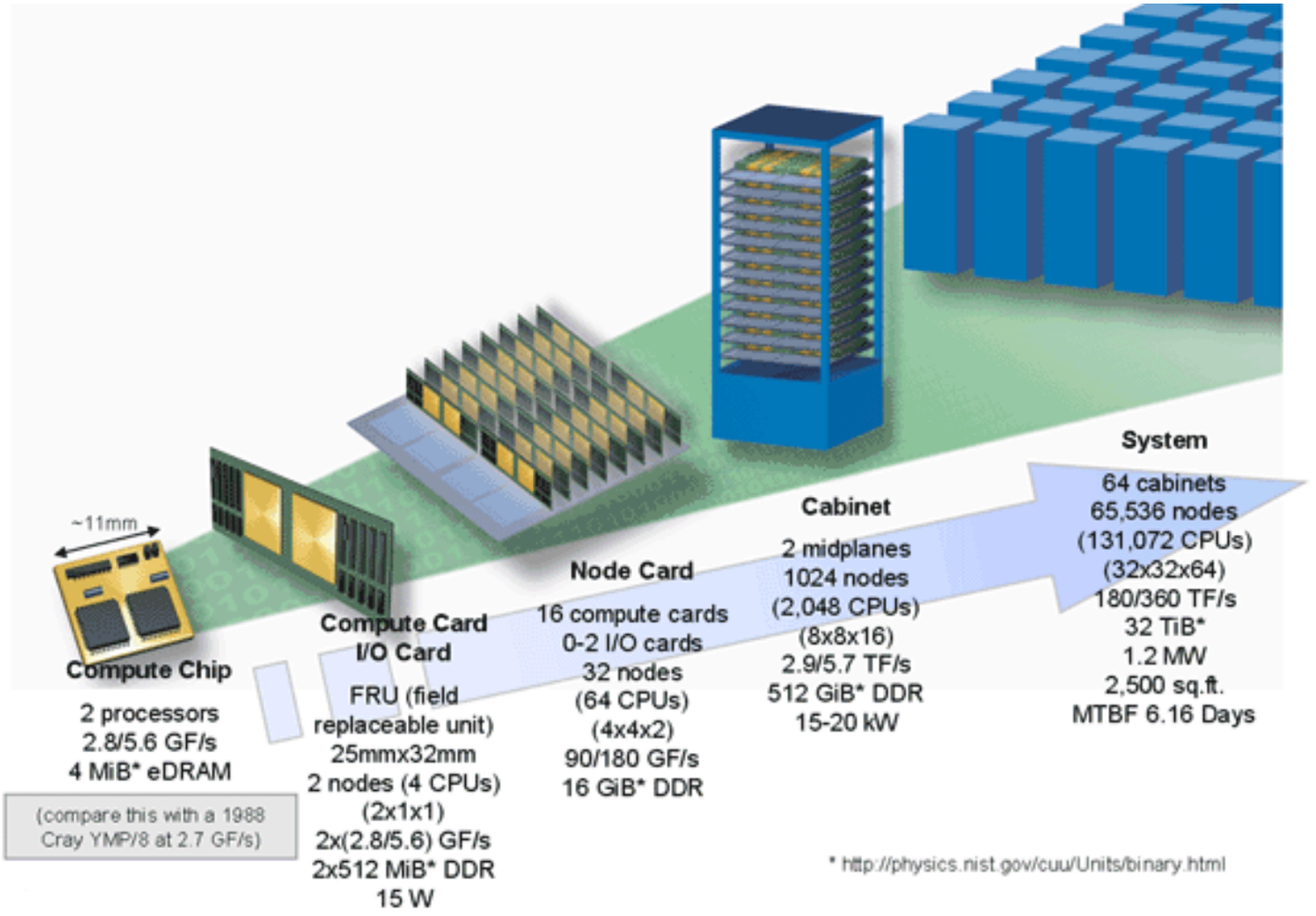
	Operations/sec	Power [Watts]	Cores	sec/cycle
Brain	10^{13}	15–20	?	10^{-3}
Intel Terascale	10^{12}	85	80	10^{-9}
Supercomputer Blue Gene	3.6×10^{14}	1.2×10^{6}	130 000	10^{-9}



- Brains work **slowly** but are **massively parallel**
- Example of various regions active at the same time

https://psychcentral.com/lib/what-is-functional-magnetic-resonance-imaging-fmri

Parallel computer example: IBM Blue Gene



https://en.wikipedia.org/wiki/IBM_Blue_Gene

Reasons to parallelize

Universität Konstanz

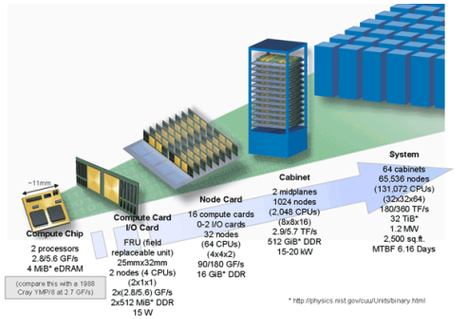
- **Save time and/or money**
  - throw compute resources at a problem and shorten compute time
  - can use cheap commodity components to build parallel processors
- **Solve larger, more complex problems**
  - some problems are so large they are infeasible to solve using serial computation, especially with limited memory
  - example: web search engines and databases processing millions of transactions every second
- **Provide concurrency**
  - a single compute resource can only do one thing at a time; multiple compute resources can do many things simultaneously

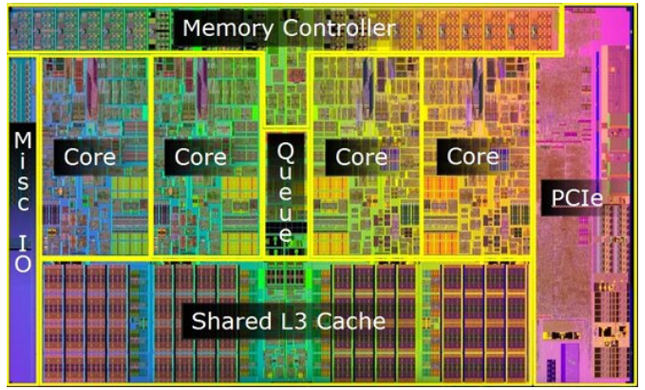
Types of parallelization

Universität Konstanz

Different ways to “multiply” processing:

- **multiprocessor:** more than one CPU
- **multicore:** one CPU with several cores
- **multithreading:** a single core executes more than one task concurrently





Process

CODE DATA FILE POINTERS

thread 1	thread 2	thread 3
registers · stack	registers · stack	registers · stack

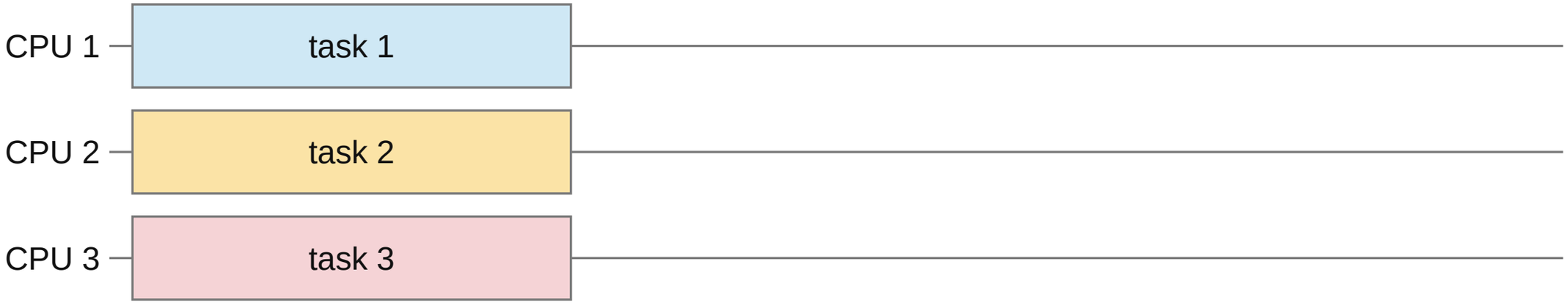
Single-thread vs Parallel

Universität Konstanz

Single-thread



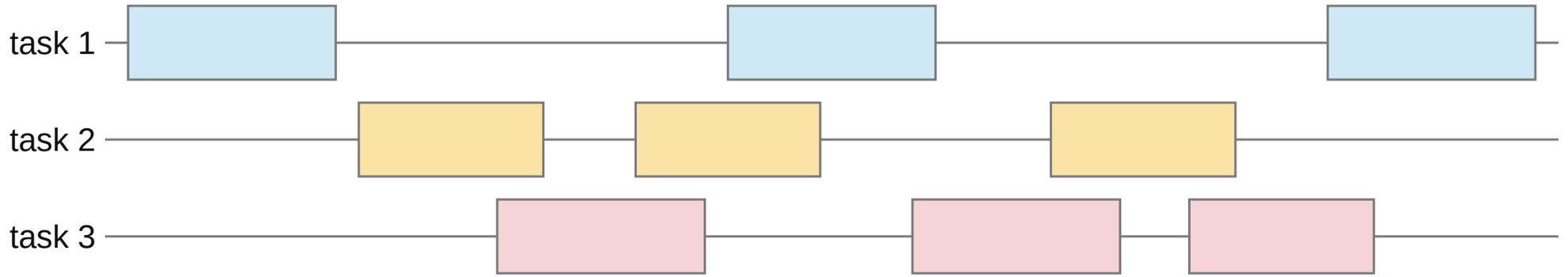
Parallel



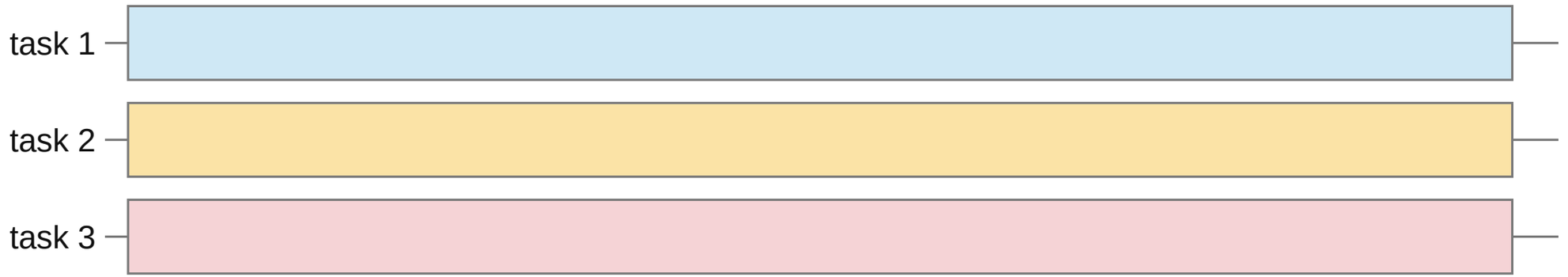
Concurrent vs Parallel

Universität Konstanz

Concurrent



Parallel



Concurrent vs Parallel

Universität Konstanz

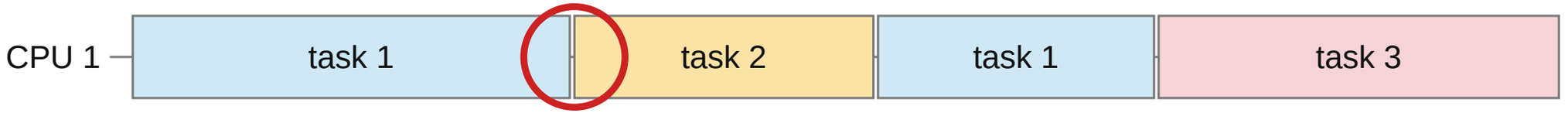
Concurrency

- Concurrency means executing multiple tasks **at the same time but not necessarily simultaneously**
- An application is making progress on more than one task at the same time (concurrently). It does not completely finish one task before it begins the next
- Tasks on the **same CPU**, order not completely fixed

Parallelism

- Parallelism means that an application splits its tasks into smaller subtasks which can be processed in parallel
- Tasks on **different CPUs**, tasks are at certain times executed in parallel

Concurrency costs and advantages



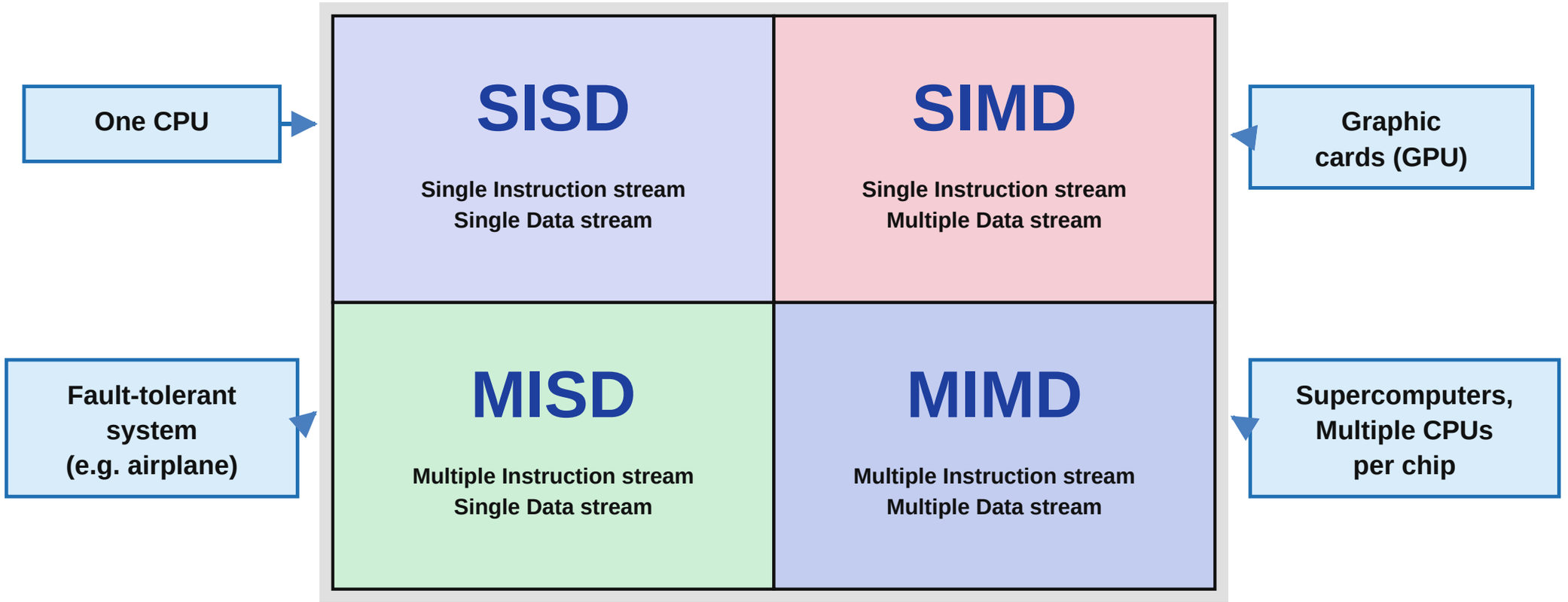
- **Switching between tasks takes computing time**
  - e.g. loading data or instructions into memory
  - this is handled by the operating system, using that compute time
- **Concurrency can save time** when using resources that are slower than the CPU. The CPU can change to another process when a slow task starts. For example:
  - access to files on the hard drive
  - access to information over a network
  - access to data from a peripheral device
  - user input

Hardware Parallelization

- Parallelization
- **Hardware Parallelization**
- Software Parallelization
- Limitations

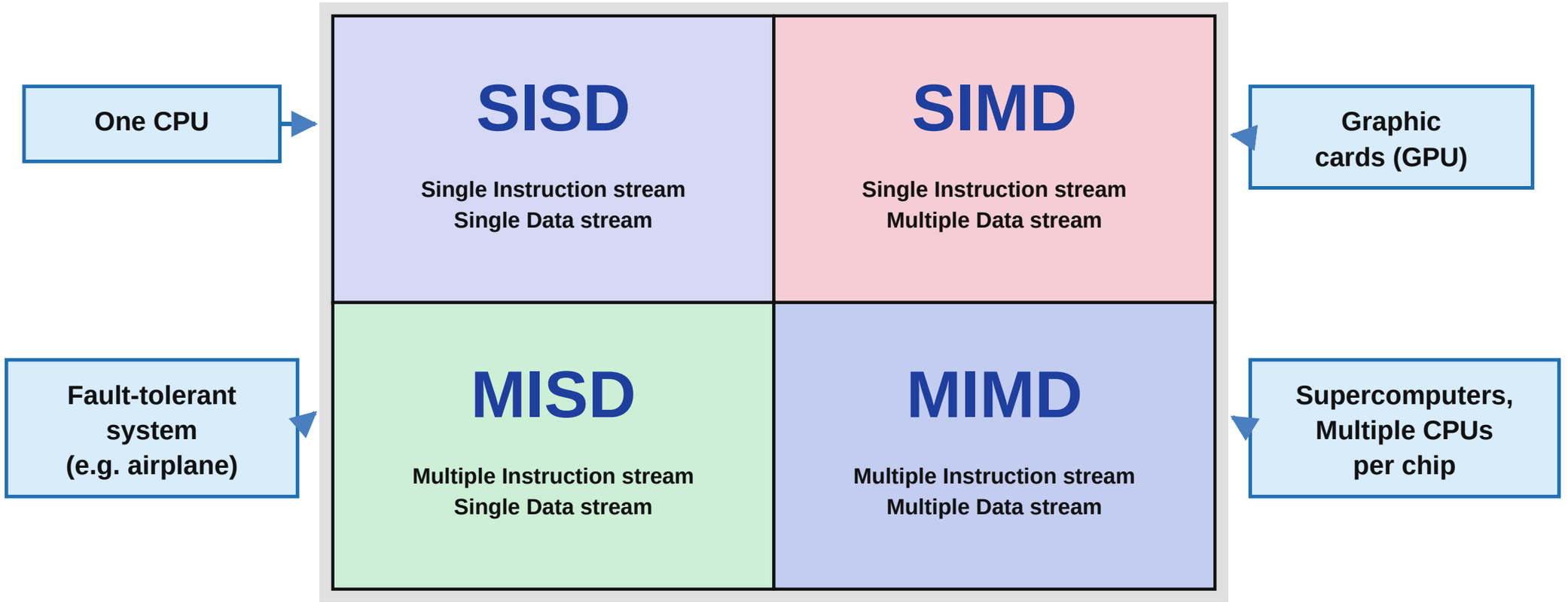
Flynn's Taxonomy

Multi-processor computer architecture (hardware) classification along two dimensions: **instruction stream** and **data stream**. Both can be single or multiple.



Flynn's Taxonomy

Multi-processor computer architecture (hardware) classification along two dimensions: **instruction stream** and **data stream**. Both can be single or multiple.



Instruction-level parallelism



- CPU architectures have instructions that are divided into **microinstructions**
- Each microinstruction takes a hardware clock cycle
- The example is an architecture with five microinstructions

Instruction-level parallelism



- CPU architectures have instructions that are divided into **microinstructions**
- Each microinstruction takes a hardware clock cycle
- The example is an architecture with five microinstructions
  - **IF:** instruction fetch – get next instruction code

Instruction-level parallelism



- CPU architectures have instructions that are divided into **microinstructions**
- Each microinstruction takes a hardware clock cycle
- The example is an architecture with five microinstructions
  - **IF:** instruction fetch – get next instruction code
  - **ID:** instruction decode – convert code into hardware activation flags

Instruction-level parallelism



- CPU architectures have instructions that are divided into **microinstructions**
- Each microinstruction takes a hardware clock cycle
- The example is an architecture with five microinstructions
  - **IF:** instruction fetch – get next instruction code
  - **ID:** instruction decode – convert code into hardware activation flags
  - **EX:** execute – perform calculation based on register contents

Instruction-level parallelism



- CPU architectures have instructions that are divided into **microinstructions**
- Each microinstruction takes a hardware clock cycle
- The example is an architecture with five microinstructions
  - **IF:** instruction fetch – get next instruction code
  - **ID:** instruction decode – convert code into hardware activation flags
  - **EX:** execute – perform calculation based on register contents
  - **MEM:** memory access – write results to main memory if necessary

Instruction-level parallelism



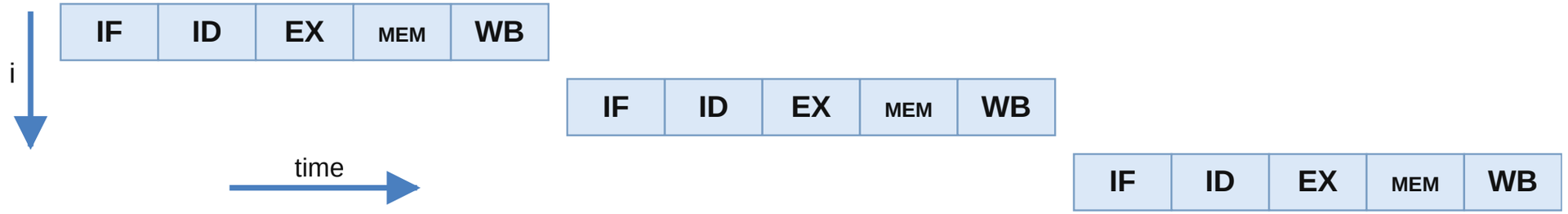
- CPU architectures have instructions that are divided into **microinstructions**
- Each microinstruction takes a hardware clock cycle
- The example is an architecture with five microinstructions
  - **IF:** instruction fetch – get next instruction code
  - **ID:** instruction decode – convert code into hardware activation flags
  - **EX:** execute – perform calculation based on register contents
  - **MEM:** memory access – write results to main memory if necessary
  - **WB:** register write back – change registers (e.g. next instruction pointer)

Instruction-level parallelism

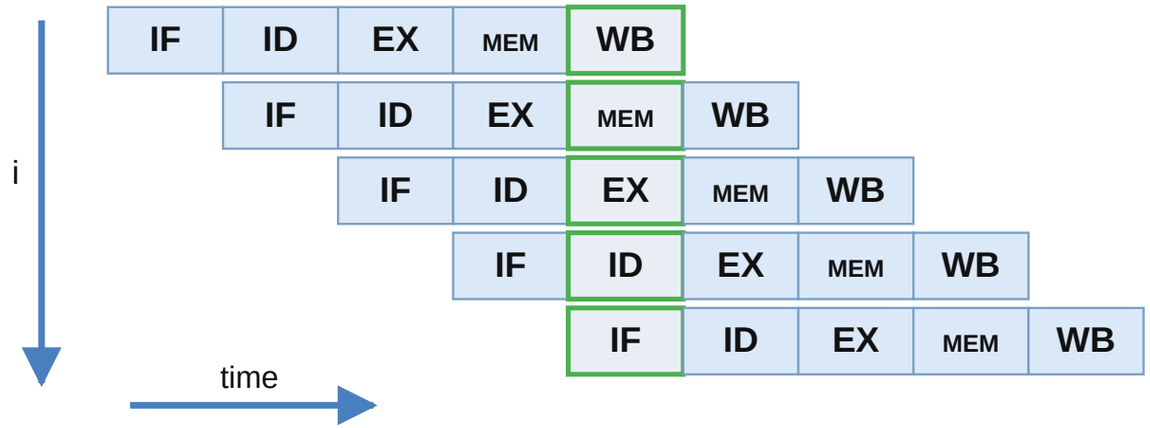


- CPU architectures have instructions that are divided into **microinstructions**
- Each microinstruction takes a hardware clock cycle
- The example is an architecture with five microinstructions
  - **IF:** instruction fetch – get next instruction code
  - **ID:** instruction decode – convert code into hardware activation flags
  - **EX:** execute – perform calculation based on register contents
  - **MEM:** memory access – write results to main memory if necessary
  - **WB:** register write back – change registers (e.g. next instruction pointer)
- Not all instructions need all five cycles. Details depend on the architecture
- Different parts of the hardware are used for each microinstruction

Instruction-level parallelism

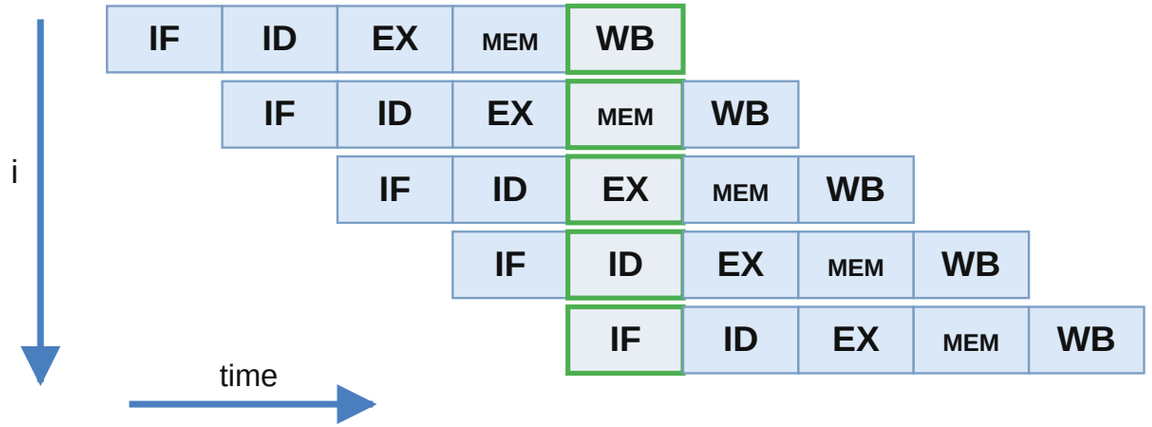


- A canonical processor **without** parallelism: takes 5 clock cycles per instruction



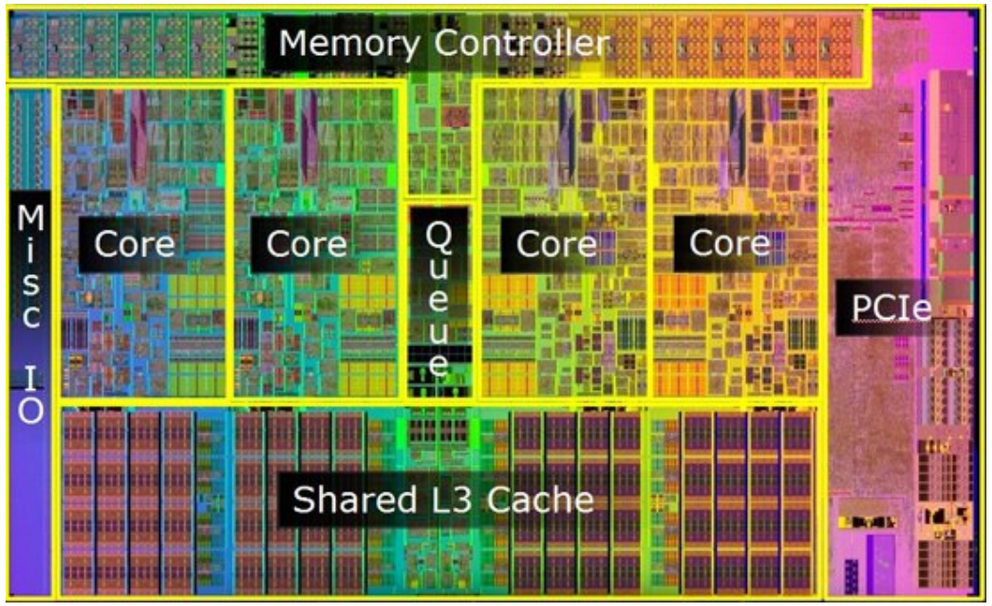
- A **pipelined** processor can simultaneously execute five instructions
- It can take up to only **one clock cycle per instruction**

Instruction-level parallelism: caveats



- **Speculative execution:** future instructions start before we finish one
  - control flow usually makes the CPU throw away future instructions
  - e.g. an `if` statement changes the next instruction → need to clean the pipeline
- **Security risks:** speculative execution leaves traces that can be read by other programs
  - the **Spectre** and **Meltdown** vulnerabilities make it possible to read private data of other programs left in the processor cache by speculatively executed instructions

Multi-core processors: your laptop

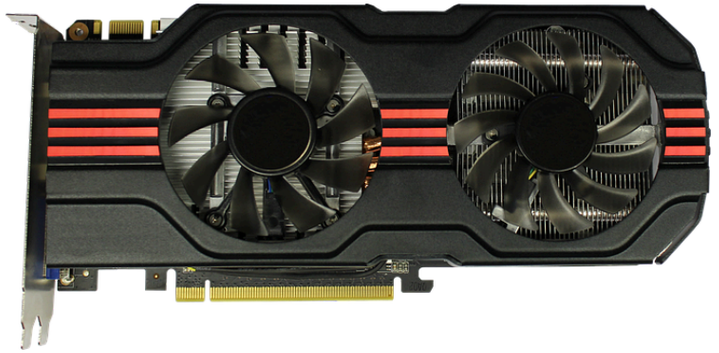


- Several cores per processor allow executing multiple threads in parallel
- Most likely in this course, all student laptops and mobile phones are multi-core and many even multi-processor

https://superuser.com/questions/584900/how-distinguish-between-multicore-and-multiprocessor-systems

Graphics Processing Unit (GPU)

- Graphics Processing Units are specialized to calculate what is displayed on a screen
- Designed for **multiplications of matrices** to calculate how to display 3D models
- Performs the same calculation over a large set of data (a vector)
- GPUs have large memories with very high bandwidth
- **Cannot do every operation efficiently!** Designed for matrix operations
- Applications beyond graphics:
  - mining cryptocurrencies: solving modular algebra equations for many possible solutions
  - training machine learning models: applying matrix operations to fit many parameters on large datasets



Computing clusters

Universität Konstanz

- **Computing cluster:** linking many computers with very high bandwidth connections
- Example: the Scientific Compute Cluster (SCC) of Uni Konstanz
  - Infiniband: 56 Gbit/s, 0.7 µs latency
  - Intel Broadwell HPC compute nodes
    - 28 cores per node
    - up to 1024 GB RAM
  - large RAID (redundant array of independent disks) memory
- The Politics & Public Administration department is an SCC **shareholder** with 4 compute nodes, i.e. 112 cores
  - part of the larger SCC infrastructure
  - if available, can use compute nodes of other shareholders. Resources might be idle when other departments do not need them

Software Parallelization

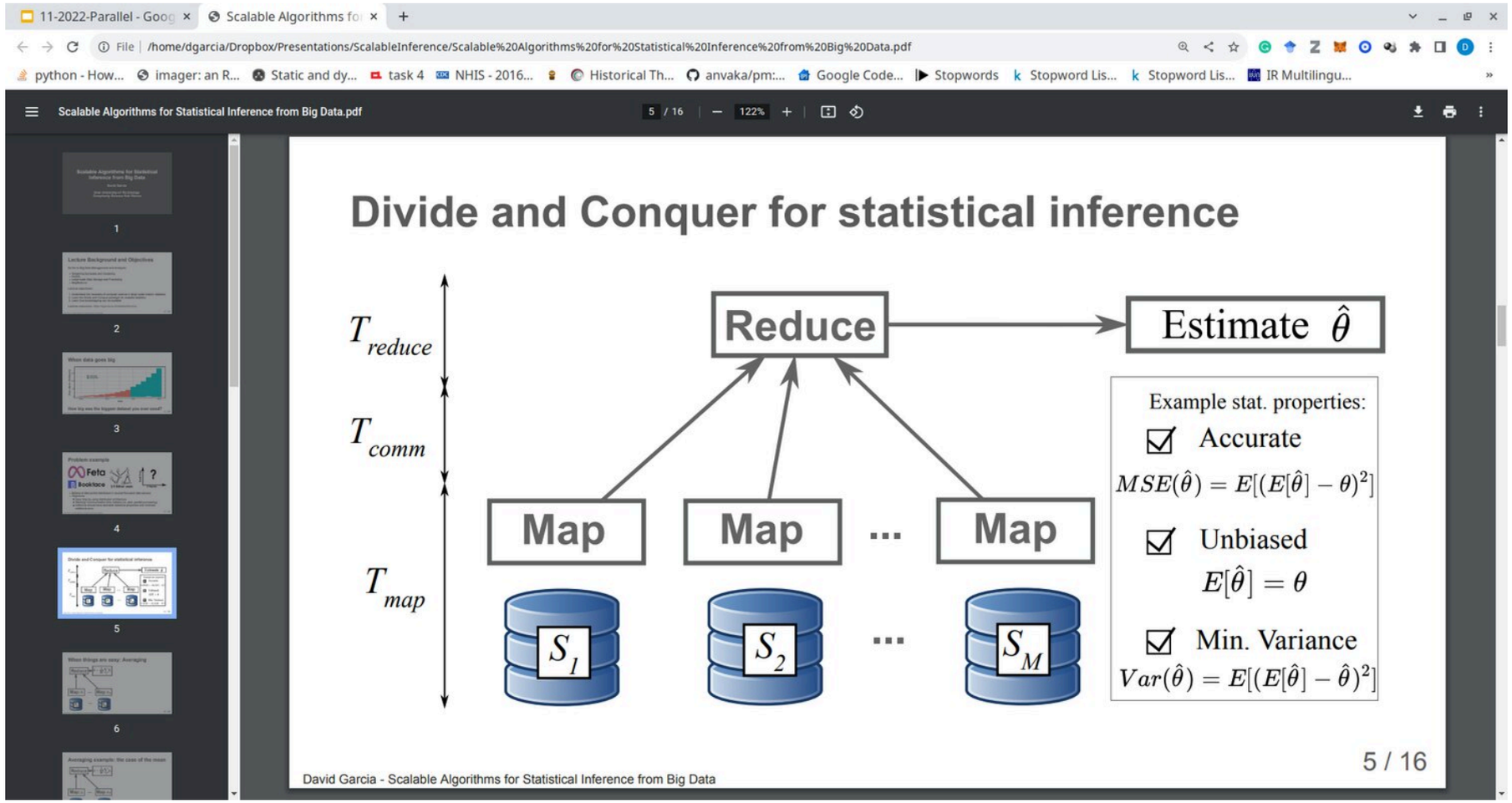
Universität Konstanz

- Parallelization
- Hardware Parallelization
- **Software Parallelization**
- Limitations

Types of Software Parallelization

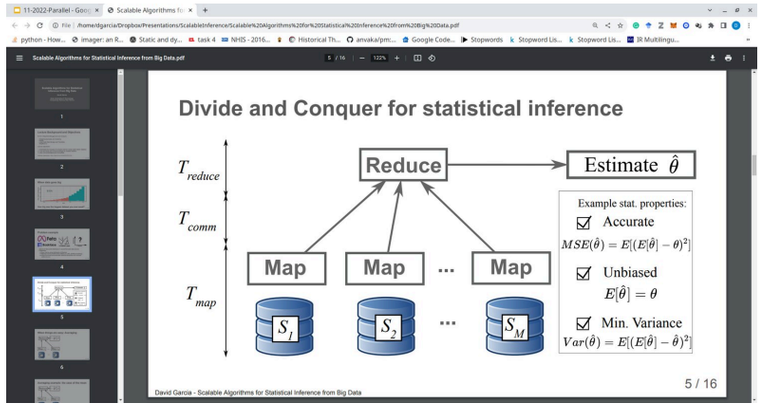
- **Multiple threads or processes in the same computer**
  - e.g. running mergesort with calls in separate parts of memory
  - *Issues:* coordinating memory access
  - *Solutions:* Inter-Process Communication (IPC) and Message Passing Interfaces (MPI) – not covered here
- **Multiple machines connected through a network: distributed computing**
  - e.g. splitting parts of mergesort to different machines and merging results in more central machines
  - *Issues:* network communication is the bottleneck for computation
  - *Solutions:* distributed paradigms like **MapReduce**

MapReduce

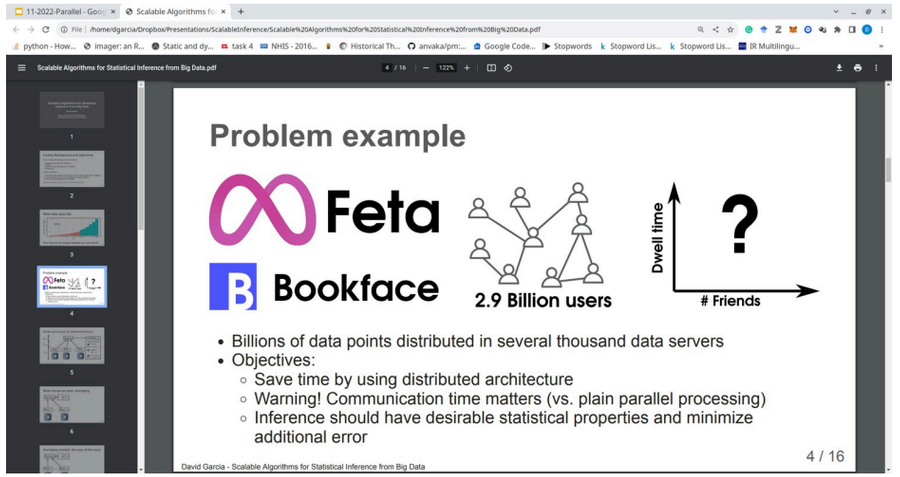

11-2022-Parallel - Goog
Scalable Algorithms fo
File | /home/dgarcia/Dropbox/Presentations/ScalableInference/Scalable%20Algorithms%20for%20Statistical%20Inference%20from%20Big%20Data.pdf
python - How...
imager: an R...
Static and dy...
task 4
NHIS - 2016...
Historical Th...
anvaka/pm:...
Google Code...
Stopwords
Stopword Lis...
Stopword Lis...
IR Multilingu...
Scalable Algorithms for Statistical Inference from Big Data.pdf
5 / 16
122%
Divide and Conquer for statistical inference
T_{reduce}
T_{comm}
T_{map}
Reduce
Estimate $\hat{\theta}$
Map
Map
...
Map
S_1
S_2
...
S_M
Example stat. properties:
Accurate
$MSE(\hat{\theta}) = E[(E[\hat{\theta}] - \theta)^2]$
Unbiased
$E[\hat{\theta}] = \theta$
Min. Variance
$Var(\hat{\theta}) = E[(E[\hat{\theta}] - \hat{\theta})^2]$
5 / 16
David Garcia - Scalable Algorithms for Statistical Inference from Big Data

MapReduce

- **Map phase:** compute parts of the solution on partial datasets in each machine
- **Communication phase:** send Map results to the Reducer over the network
- **Reduce phase:** combine Map results into the final solution
- Computations can take several Map-Reduce iterations or hierarchies
- Apache Hadoop and Spark are some of the most popular software solutions for MapReduce

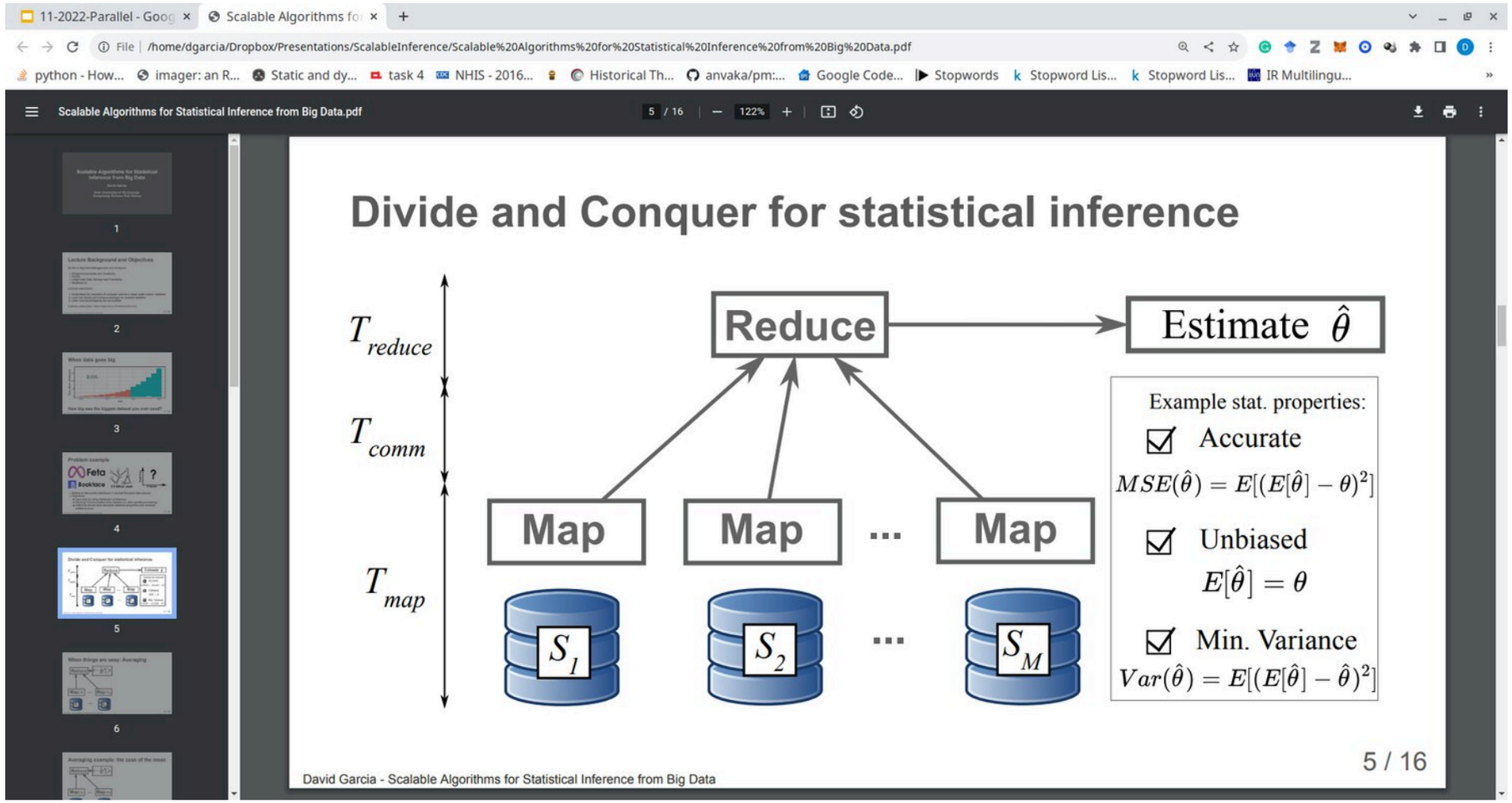


Distributed statistical inference example

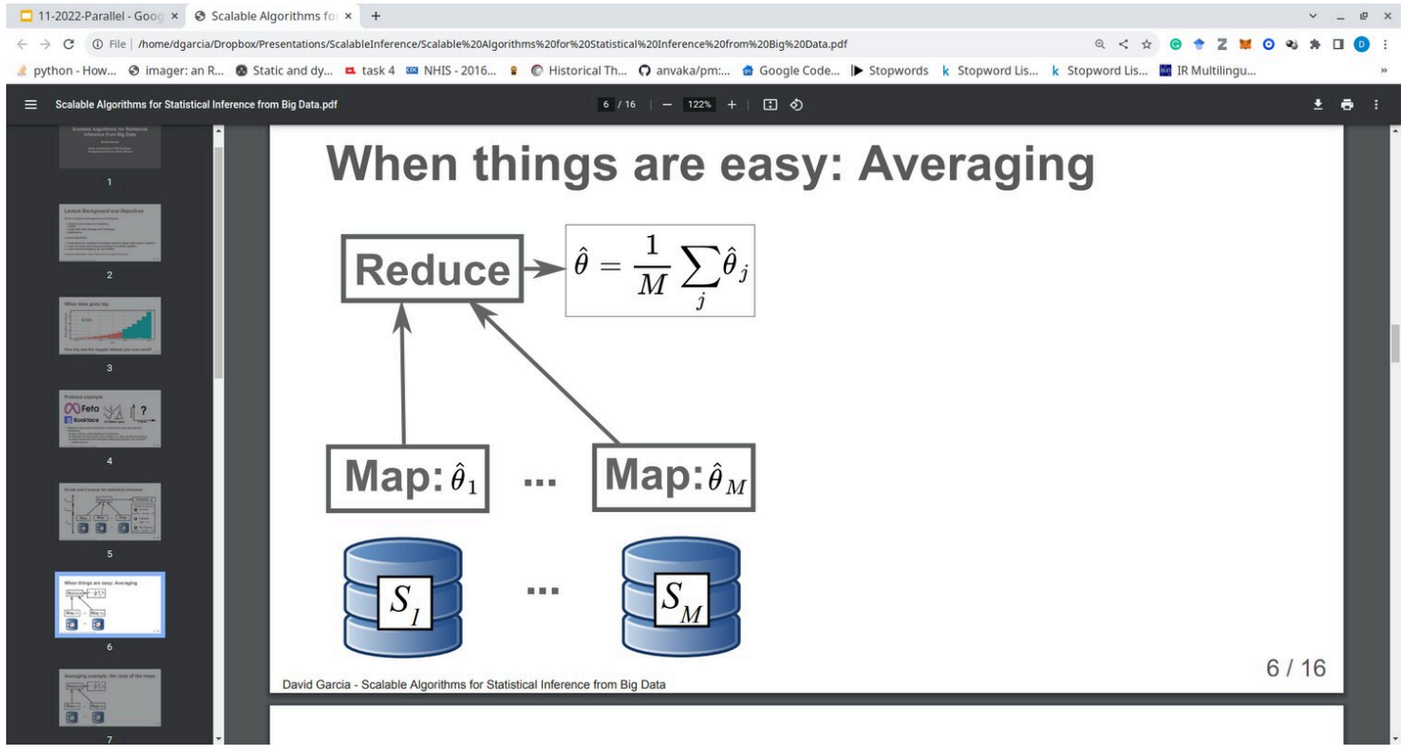


- Billions of data points distributed over several thousand data servers
- Objectives:
  - Save time by using a distributed architecture: **bring the code to the data**
  - Warning! Communication time matters (vs. plain parallel processing)
  - Inference should have desirable statistical properties and minimize additional error

Distributed statistical inference: desired properties

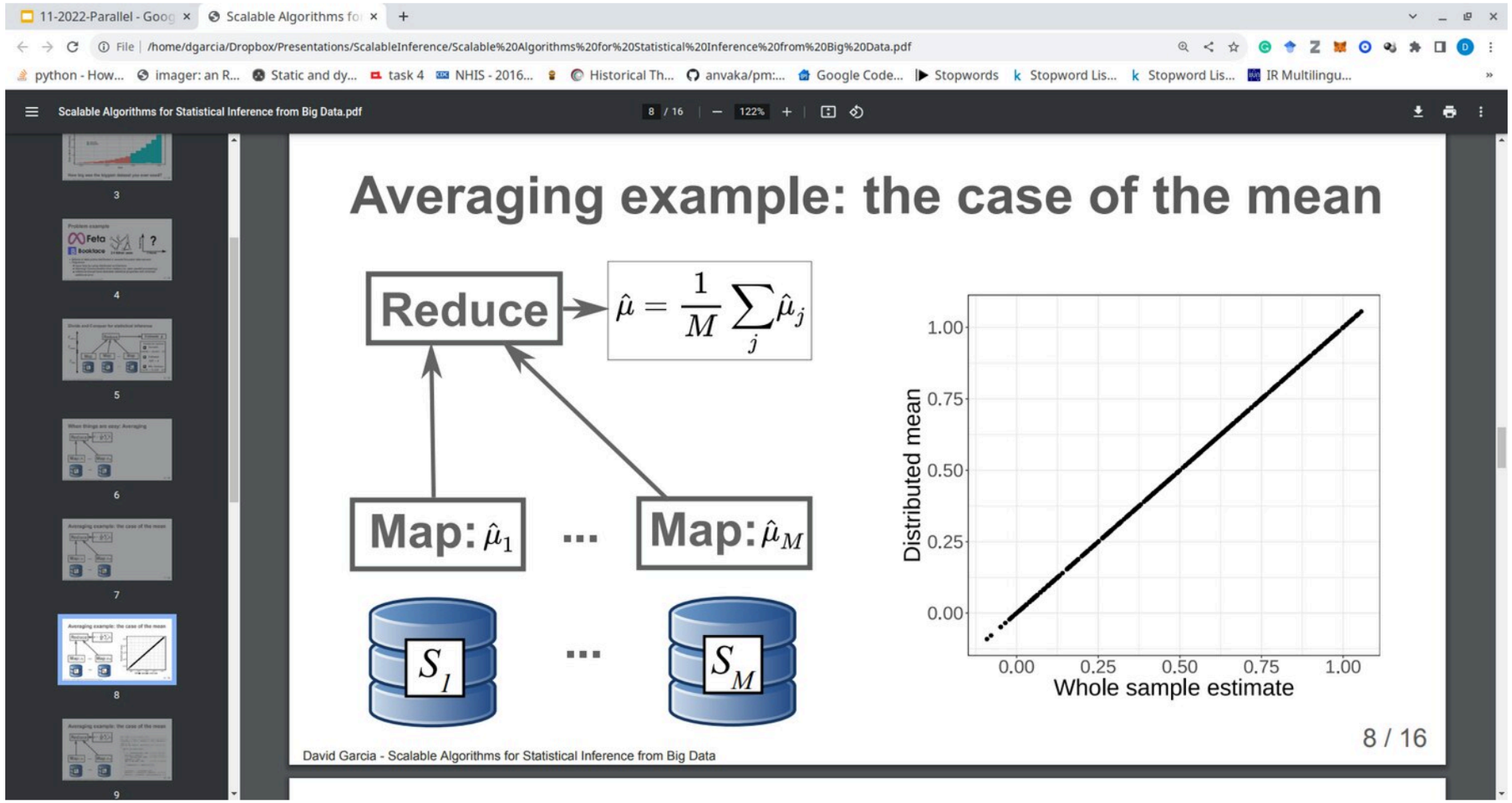


When things are easy: averaging

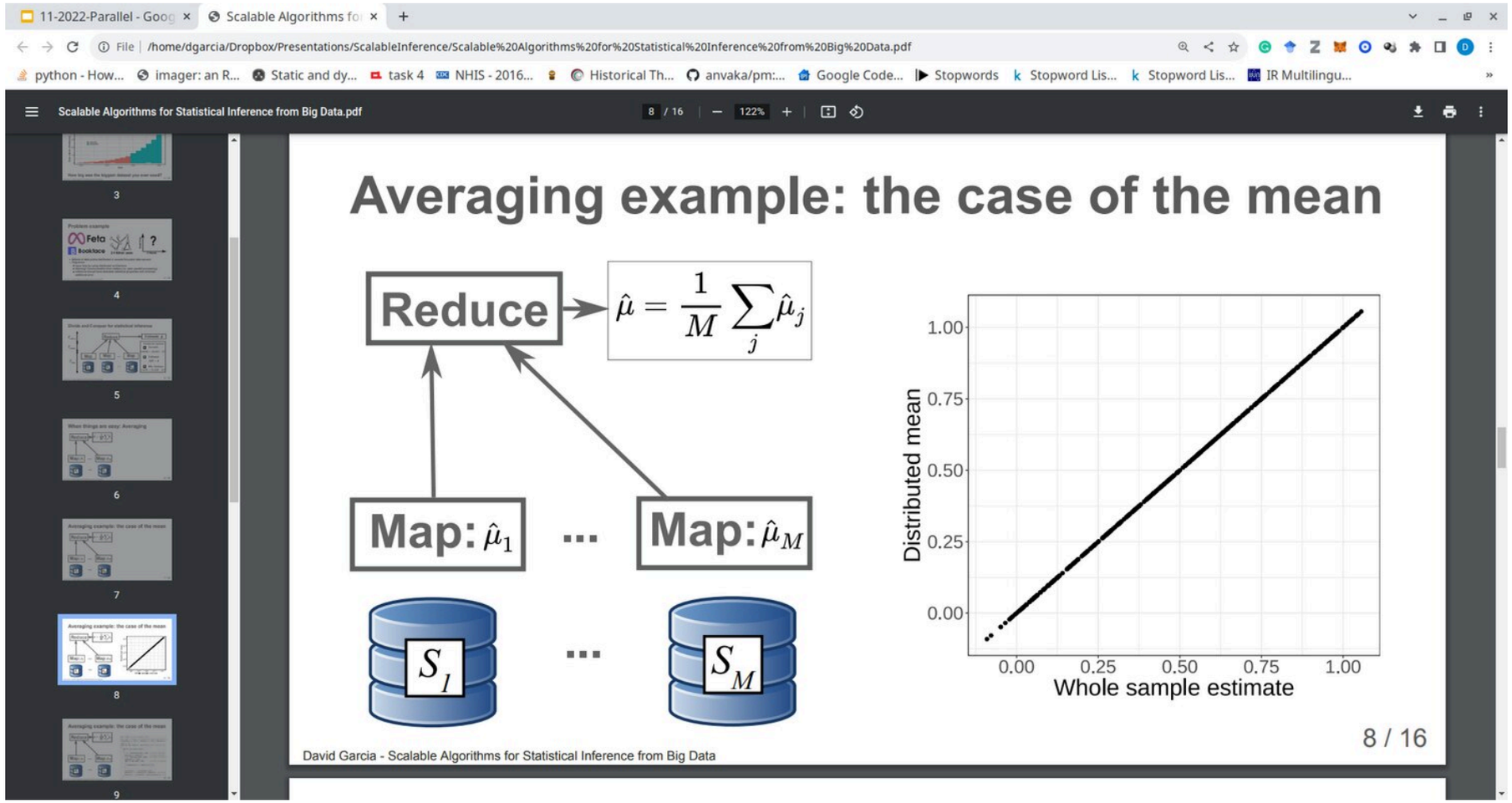


- **Map:** compute the same estimator in each local sample
- **Reduce:** aggregate to the final estimator by averaging each Map result

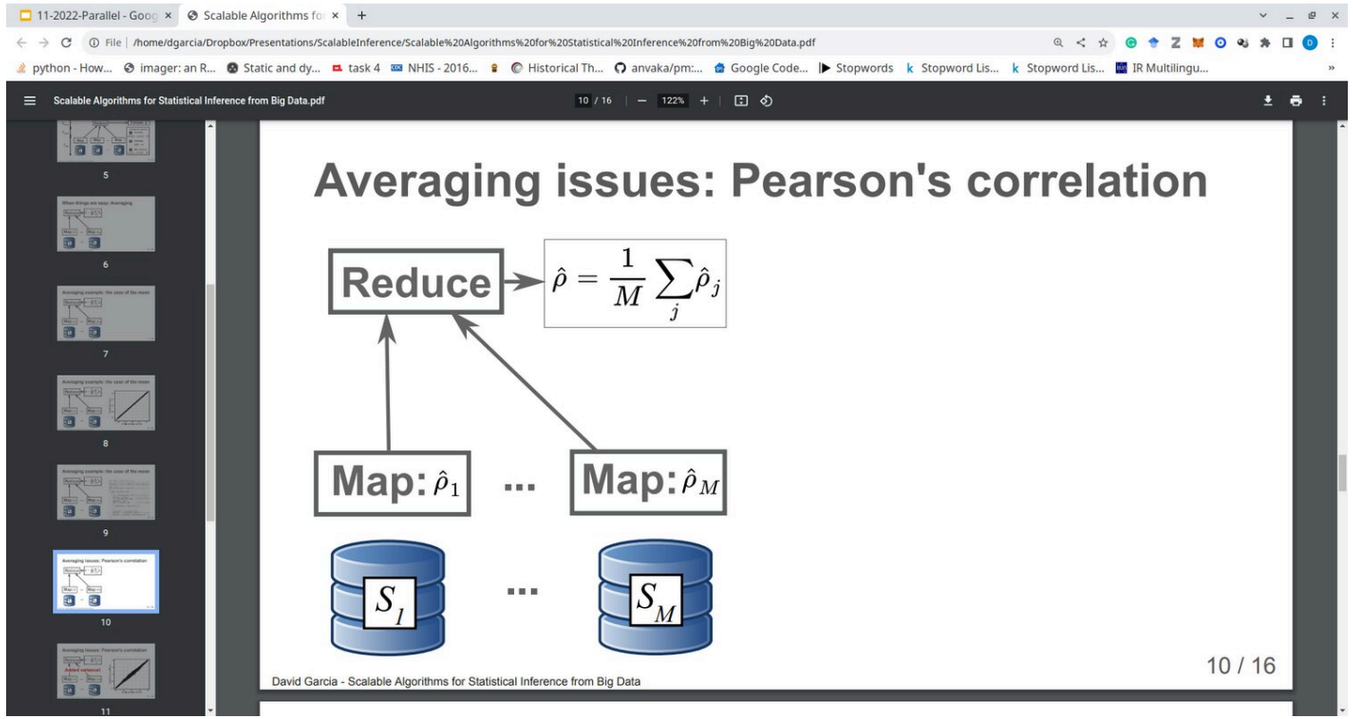
Averaging: computing a mean



Averaging: computing a mean



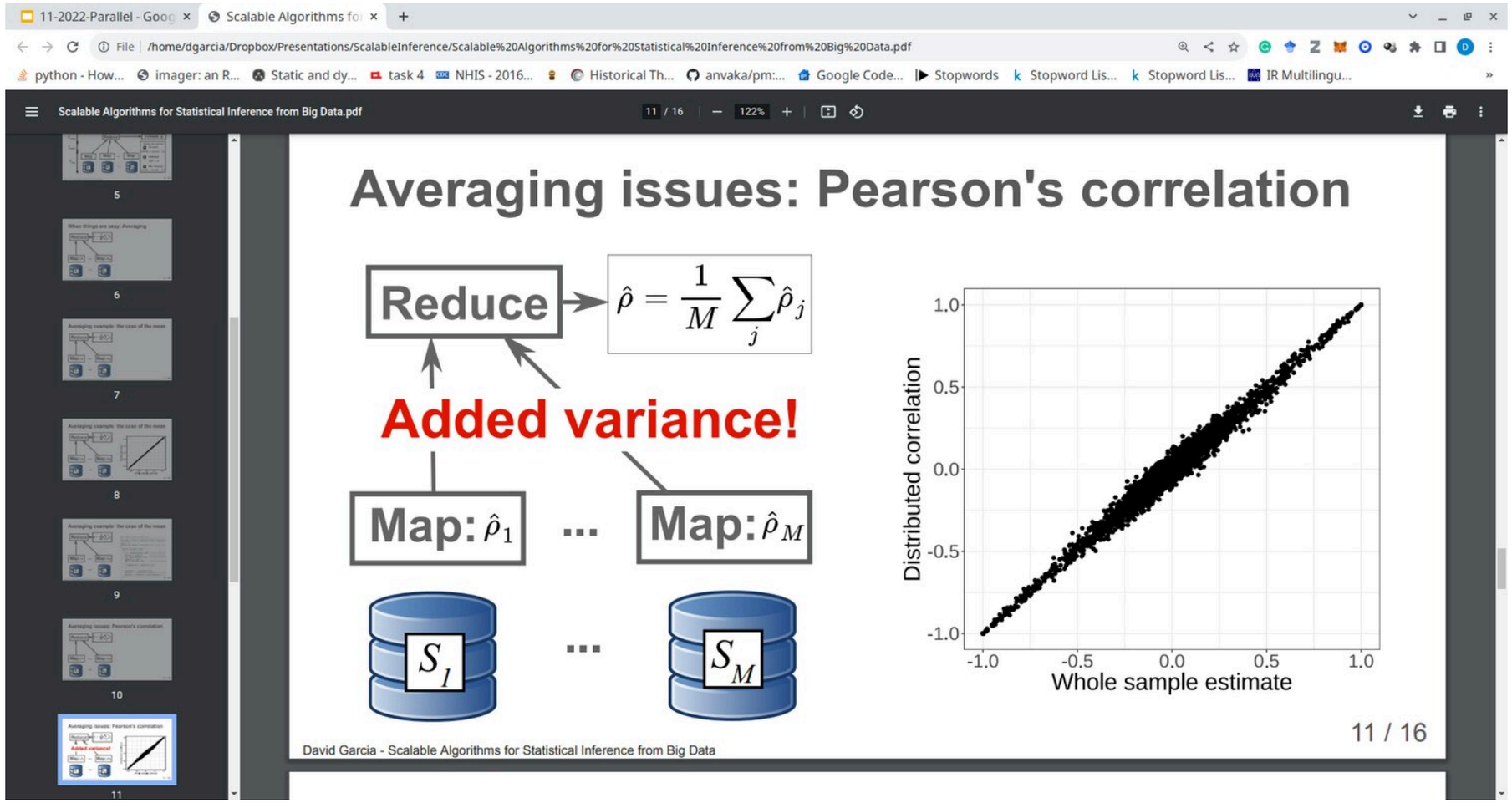
Problems with averaging: Pearson's correlation



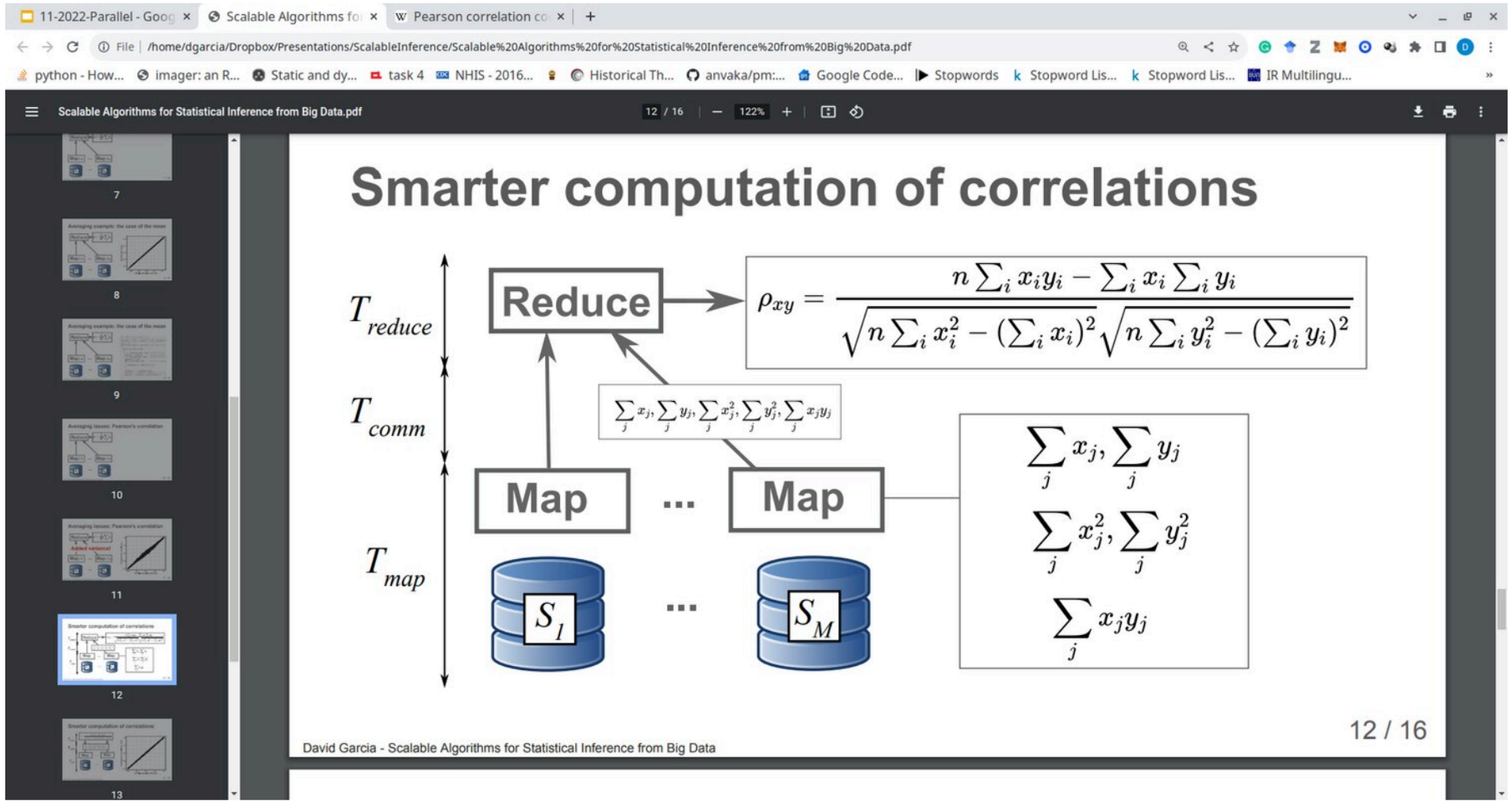
- **Map:** compute Pearson's correlation in each local sample
- **Reduce:** aggregate to the final estimator by averaging each Map result

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

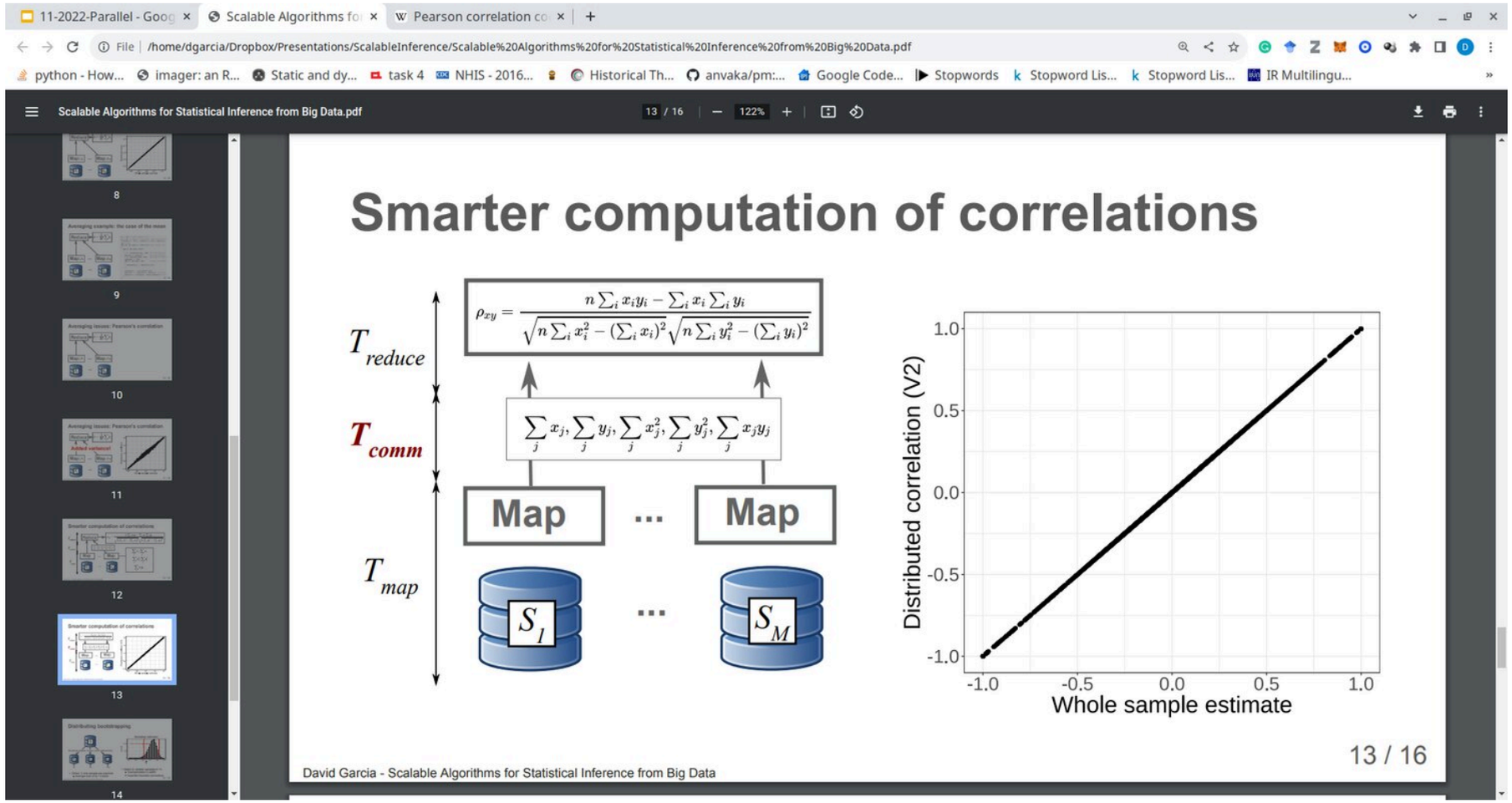
Problems with averaging: Pearson's correlation



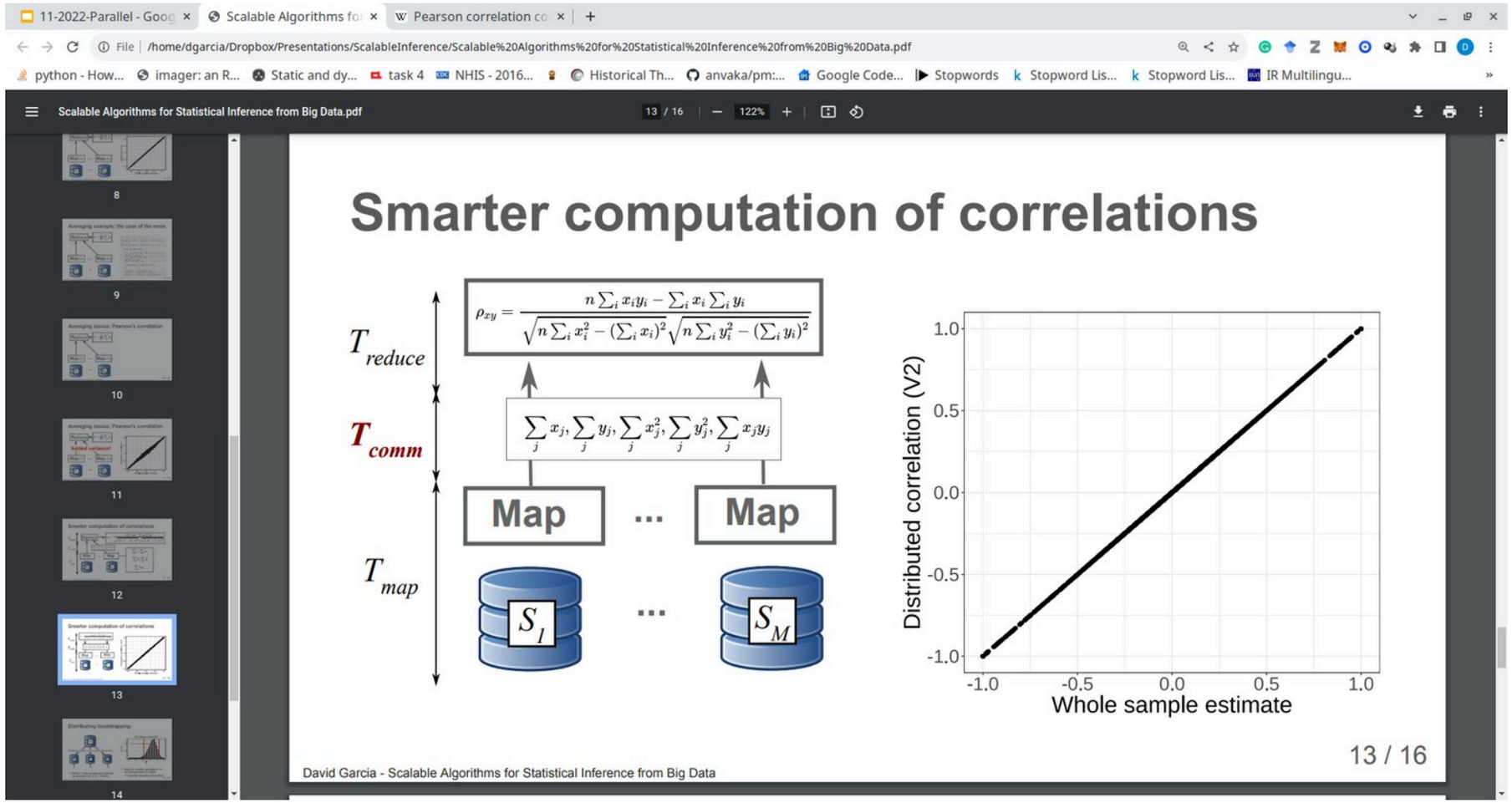
Smarter distributed calculation of correlations



Smarter distributed calculation of correlations



Smarter distributed calculation of correlations



Added communication time tradeoff to avoid added variance in the estimate

Limitations

- Parallelization
- Hardware Parallelization
- Software Parallelization
- **Limitations**

Limits to Parallelization

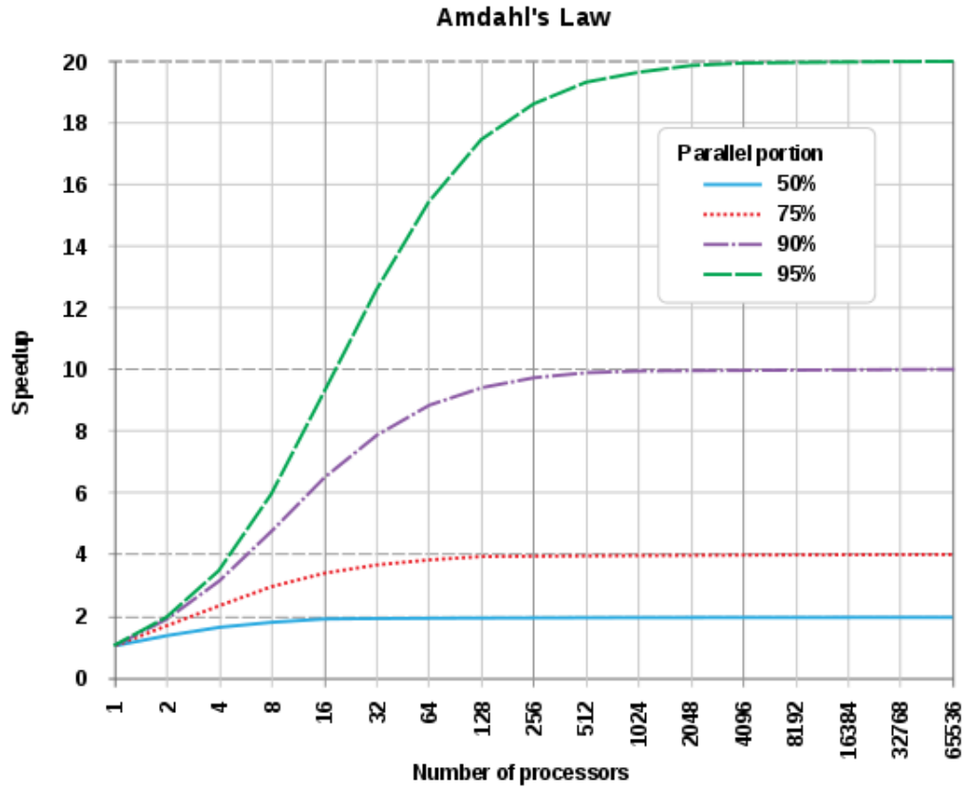
Universität Konstanz

- Can you actually achieve an optimal speedup?
- **Gene Amdahl (1967): there is an upper limit**
- Intuition:
  - there is always a part of the code that **cannot** be parallelized
  - with increasing parallelization, this part dominates the overall runtime of the program
- Maximal speedup depends on two variables:
  - P: fraction of the computation that can be parallelized, e.g. 80%
  - N: number of processors / threads that can be used in parallel

Limits to Parallelization: Amdahl's law

$$S = \frac{1}{(1 - P) + \frac{P}{N}}$$

- S: speed-up of the execution of the whole task, e.g. 2×
- P: fraction of the computation that can be parallelized, e.g. 80%
- N: number of processors / threads that can be used in parallel



Summary

Universität Konstanz

Now: course surveys!

- **Parallelization**
  - Motivation with Moore's law
  - Example paradigms: multi-core, multi-processor, multi-thread
  - Parallelization vs concurrency
- **Hardware Parallelization**
  - Instruction-level parallelization
  - Multi-core processors, GPUs and clusters
- **Software Parallelization**
  - Distributed computing: MapReduce
- **Limitations:** Amdahl's law

Next session: Social Science Applications

Please help us to improve the course!

- Survey only for the **Theory** part of ICSS
- https://evasys.uni-konstanz.de/evasys/online.php?pswd=ATEVY
- It takes about 5-10 minutes
- Completely anonymous



Please help us to improve the course!

- Survey only for the **Practice** part of ICSS
- https://evasys.uni-konstanz.de/evasys/online.php?pswd=XZ7GC
- It takes about 5-10 minutes
- Completely anonymous

